Universidad Abierta Interamericana

# PROGRAMACIÓN ORIENTADA A OBJETOS

**CÓDIGO DE MATERIA 11**

**Titular: Darío Guillermo Cardacci**

**Alumna: Morana, Valentina**

## FACULTAD DE TECNOLOGÍA INFORMÁTICA

## UNIVERSIDAD ABIERTA INTERAMERICANA

# ÍNDICE

GUÍA DE TRABAJOS PRÁCTICOS..... 3

RESOLUCIÓN DE LAS ACTIVIDADES..... 6

GUÍA DE ABORDAJE BIBLIOGRÁFICO.....28

# GUÍA DE TRABAJOS PRÁCTICOS

## UNIDAD I

1. Desarrollar una jerarquía de clases relacionadas por herencia y agregación que represente la estructura necesaria para identificar claramente los elementos encontrados en un sistema de calificaciones de una universidad. En cada clase detallar métodos y propiedades.

2. Defina una situación donde se desee llegar desde un estado inicial a uno final, por ejemplo, una cuenta bancaria que posee un saldo inicial y se la somete a una operación bancaria. Desarrollar la clase cuenta con las propiedades y métodos que considere pertinentes y demuestre cómo cambia el estado del objeto en la medida que se le realizan depósitos, extracciones y transferencias.

3. Genere una estructura de clases donde se pueda observar la herencia simple y se justifique utilizar el polimorfismo.

4. Genere una estructura de clases donde se pueda observar la herencia múltiple y se justifique utilizar el polimorfismo.

5. Desarrollar conjuntos de objetos reales donde para su agrupación y clasificación sea necesario aplicar la categorización conceptual, la clásica y el agrupamiento prototípico. Justifique.

## UNIDAD II

6. Desarrollar un programa que posea una estructura de clases donde se puedan observar dos métodos y el constructor sobrecargados.

7. Desarrollar un programa que posea una estructura de clases donde se pueda observar una propiedad de solo lectura, una propiedad de solo escritura, una propiedad de escritura- lectura, una propiedad de predeterminada y una propiedad con argumentos.

8. Desarrollar un programa que posea una clase que aplique un destructor que finalice el ciclo de vida de un objeto que se instranció en el constructor.

9. Desarrollar un programa que posea una clase que tenga propiedades, métodos y sucesos. Los sucesos deben tener suscripciones manuales realizadas por el programador a funciones de la clase donde se encuentra instanciado el objeto que posee los sucesos.

10. Desarrollar un programa que posea una clase que contenga al menos dos campos, tres métodos, un constructor y dos sucesos estáticos. Comente que característica le otorga esta característica a cada miembro.

11. Desarrollar un programa donde se observe claramente el uso de los miembros de base en un entorno de herencia. Enfatice las particularidades al usarlo en los constructores y

finalizadores. Demuestre en el mismo programa el uso de this. Establezca las diferencias y en qué caso se justifica utilizar cada uno.

12. Desarrollar un programa que posea al menos una clase abstracta, un método virtual, una clase sellada y una clase anidada.

## UNIDAD III

13. Desarrollar un programa que genera varias instancias (una cantidad importante), verifique la memoria utilizada, pase el GC y vuelva a verificar el espacio de memoria. ¿Qué se observa?

14. Desarrollar un programa que genere una instancia, pierda la referencia a la misma y aplicando la técnica de "resurrección de objetos" logre obtener la referencia a ese mismo objeto.

15. Desarrollar un programa que aplique el concepto de manejo de errores. La estructura propuesta debe tener al menos 5 Catch y el finally.

16. Desarrollar un programa que aplique el concepto de manejo de errores. Generar un error personalizado por medio de una clase que herede de Exception y disparar el error con Throw.

## UNIDAD IV

17. Desarrollar un programa que implemente la interfaz IComparable. Genere un arrary de objetos y ordénelos.

18. Desarrollar un programa que implemente la interfaz IComparar. La clase que lo implementa deberá tener al menos cinco criterios de ordenamiento. Genere un arrary de objetos y ordénelos por cada criterio.

19. Desarrollar un programa que implemente la interfaz ICloneable. Clone el objeto que posee la interfaz y demuestre que la clonación funcionó.

20. Desarrollar un programa que implemente las interfaces IEnumerable e IEnumerator. Lo adaptado recórralo con el for each y muestre los resultados.

## UNIDAD V

21. Desarrollar un programa que aplique el concepto genérico a nivel de clase y en al menos dos métodos.

22. Desarrollar un programa que utilice Linq para realizar consultas.

23. Desarrollar un programa que utilice expresiones lambda.

## UNIDAD VI

24. Desarrollar un programa que utilizando socket que permita escribir en una aplicación que se esté ejecutando en una PC de la red y lo escrito se pueda visualizar en la aplicación que está corriendo en otra PC de la red.

25. Desarrollar un programa utilizando socket que emule un servidor de chat. También desarrollar el software cliente que se ejecutará en las PC's que se conecten al servidor. Cada mensaje enviado por un cliente lo podrán recibir todos los demás. En el caso que un cliente solicite comunicación privada el mensaje se verá solo por los que participan de esta comunicación. Un cliente puede solicitar armar un grupo y el resto unirse a él, para que esto suceda el propietario del grupo deberá aceptarlos y tiene la potestad de desconectarlos. El servidor es quien modera todo y por allí pasan todos los mensajes.

26. Desarrollar un programa que haciendo uso del puerto paralelo y un emulador que permita enviar señales al mismo y recolectar las entradas que se producen en él.

# RESOLUCIÓN DE LAS ACTIVIDADES

**Aclaración del Alumno**

Estos ejercicios fueron resueltos en código utilizando C# con .NET Framework. Se adjunta una breve descripción de lo realizado en cada caso y/o su correspondiente justificación según lo solicitado en el enunciado.

También se incluyen los diagramas de clases debajo de cada explicación. Los mismos fueron elaborados con PlantUML. También se encuentran disponibles en el repositorio de GitHub en formato PDF para una visualización más clara y detallada.

Repositorio GitHub: https://github.com/valentinamorana/POO_Verano

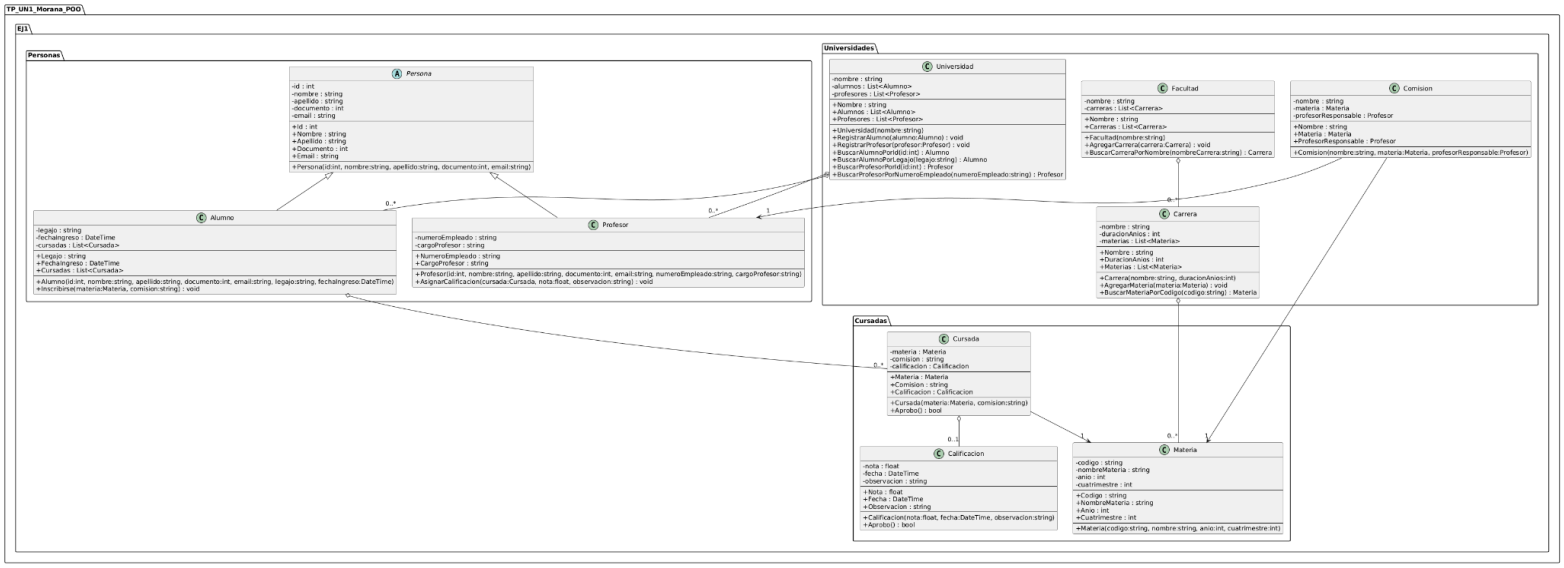
## Unidad 1

**Ejercicio 1**

El sistema modela una Universidad como clase principal que contiene listas de Carrera, Alumno y Profesor. La relación entre Universidad y estas clases es de agregación, ya que la universidad administra y agrupa estos objetos, pero ellos pueden existir independientemente.

Por otro lado, la clase Carrera se relaciona con Alumno y Profesor mediante asociación, ya que los alumnos se inscriben en carreras y los profesores dictan materias dentro de ellas.

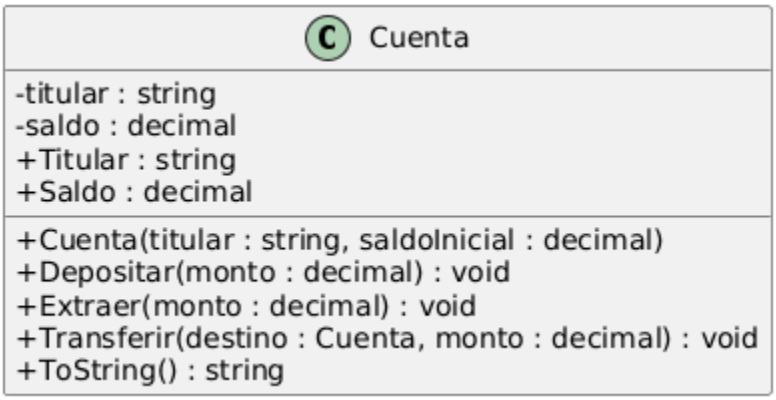
En conjunto, el modelo representa cómo los objetos se vinculan entre sí y cómo el estado del sistema se organiza a través de estas relaciones.



**Ejercicio 2**

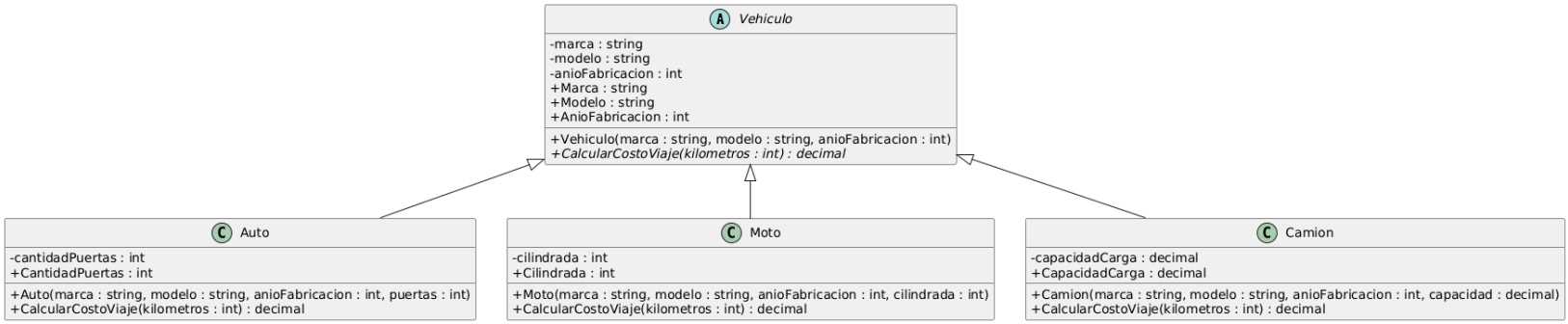
En este ejercicio se modeló una situación bancaria donde una cuenta parte de un estado inicial definido por su titular y un saldo inicial. Se desarrolló la clase Cuenta con propiedades y métodos que permiten modificar su estado a través de operaciones como Depositar, Extraer y Transferir.

Cada operación impacta directamente sobre el atributo Saldo, demostrando cómo el objeto cambia su estado interno a medida que se ejecutan distintas acciones. De esta manera, se representa el comportamiento dinámico de la cuenta bancaria, mostrando la evolución desde un estado inicial hasta distintos estados finales según las operaciones realizadas.




**Ejercicio 3**

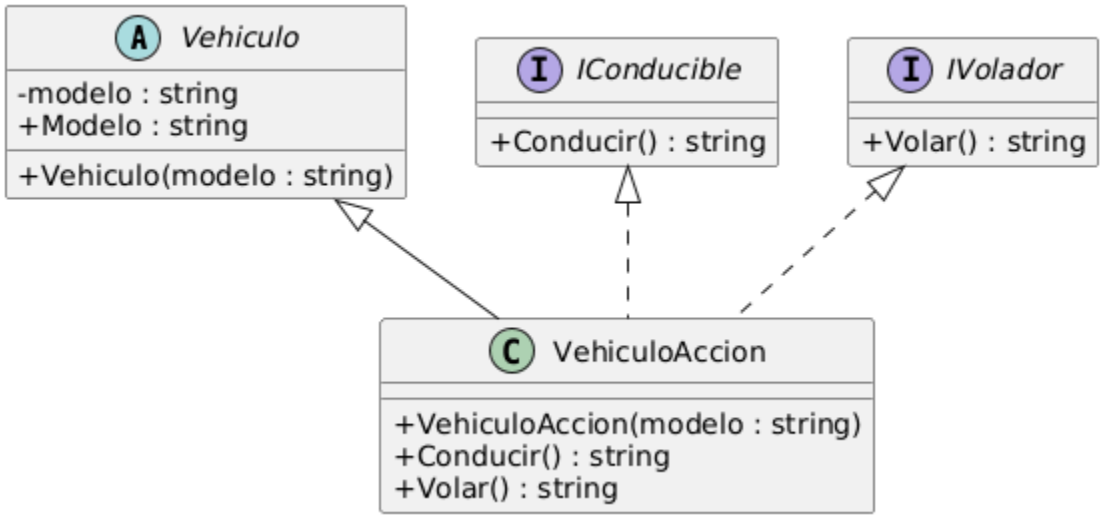
En este programa se utiliza la herencia simple al definir una clase base Vehiculo con atributos comunes (marca, modelo, año) y clases derivadas (Auto, Moto, Camion) que especializan el comportamiento. Y, se aplica polimorfismo porque todas las clases responden al método “CalcularCostoViaje”, pero cada tipo concreto lo calcula de manera distinta. Esto permite manejar colecciones de Vehiculo sin condicionales, facilitando extensibilidad y mantenimiento.



**Ejercicio 4**

C# y .Net solo admiten la herencia simple. Esto significa que una clase derivada sólo puede heredar de una única clase base.

Por eso mismo en este sistema se utiliza herencia múltiple mediante la implementación de múltiples interfaces en la clase VehiculoAccion. El polimorfismo se observa cuando el mismo objeto es tratado como IConducible o IVolador, ejecutando distintos comportamientos según el tipo de referencia.




**Ejercicio 5**

1) Conjunto: Medios de transporte (auto, moto, bicicleta, colectivo, tren, avión, monopatín)

a) Agrupamiento conceptual (por concepto/uso)

Criterio: “tipo de uso” (no es una propiedad física estricta; es un concepto que depende del contexto).

- Urbano individual: bicicleta, monopatín, moto
- Urbano masivo: colectivo, tren
- Larga distancia: tren, avión

Justificación: cuando es confuso fijar una característica única medible, pero se puede definir una descripción conceptual (por ejemplo “transporte urbano masivo”), se clasifica por conceptos. El límite no es rígido: un mismo objeto (tren) puede caer en más de un grupo según la “mayor adecuación” al concepto.

b) Categorización clásica (por propiedades necesarias y suficientes)

Regla: “motorizado” vs “no motorizado”

- Motorizado: auto, moto, colectivo, tren, avión
- No motorizado: bicicleta, monopatín (si es a fuerza humana)

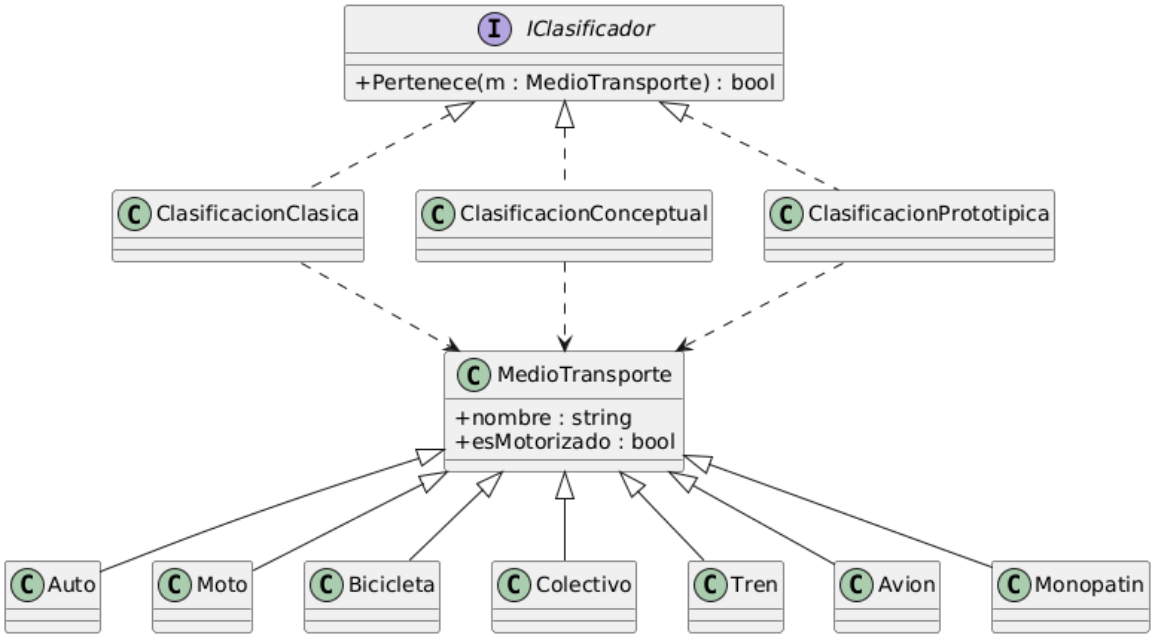
Justificación: se usan propiedades como criterio: si el objeto tiene (o no) la propiedad “motor”, pertenece o no pertenece. Son condiciones necesarias/suficientes para decidir el grupo.

c) Prototípica (por semejanza con un prototipo)

Prototipo: “auto” como vehículo típico.

- Más parecido: camioneta, taxi (variantes del auto)
- Menos parecido: moto, bicicleta
- Borderline: monopatín eléctrico (¿vehículo “como auto” o algo distinto?)

Justificación: cuando no hay una lista clara de propiedades comunes ni una descripción conceptual consistente, se usa un objeto prototípico y se decide por grado de semejanza (parecido familiar). La frontera es difusa.



2) Conjunto: Alimentos (manzana, pan, queso, yogur, pizza, ensalada, agua, gaseosa)

a) Agrupamiento conceptual

Criterio: “rol en la comida” (concepto social/cultural)

- Desayuno/merienda: yogur, pan
- Plato principal: pizza, ensalada
- Bebidas: agua, gaseosa

Justificación: la categoría está dada por una descripción conceptual (“comida de desayuno”, “plato principal”), que depende del contexto y no de una propiedad única estricta.

b) Categorización clásica

Reglas (propiedades):

- “Bebida” = se consume en estado líquido → agua, gaseosa
- “Fruta” = fruto de planta frutal → manzana

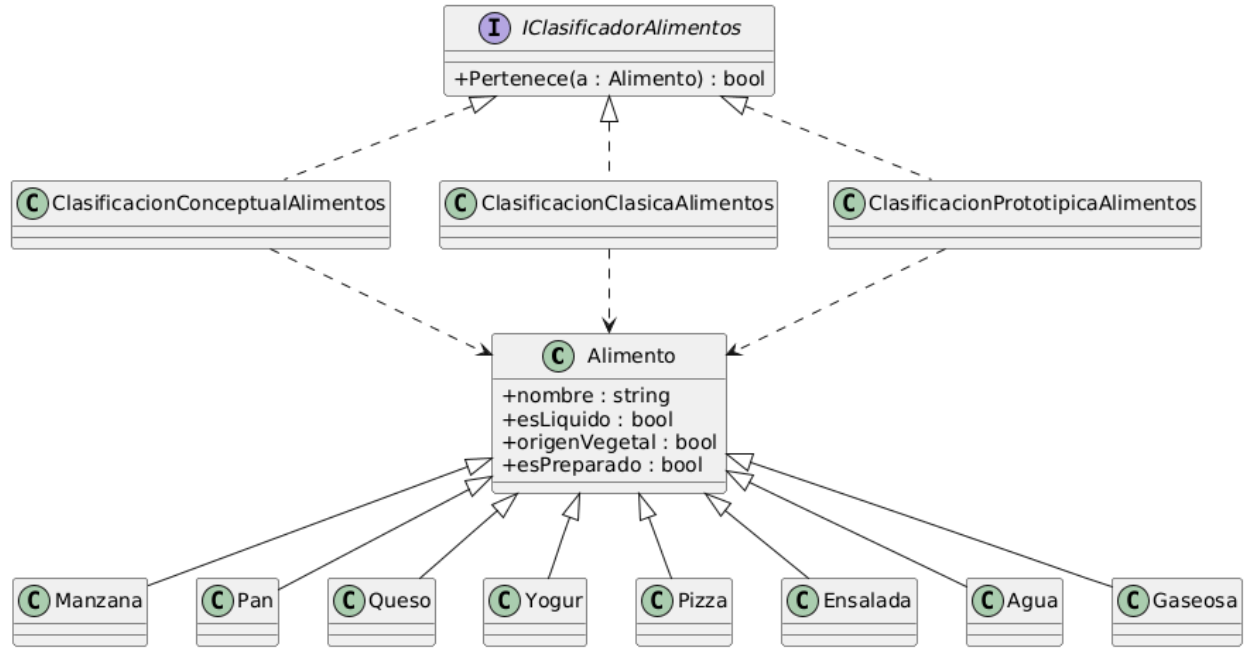
Justificación: se define la categoría por propiedades necesarias/suficientes (estado líquido; origen como fruto). Si cumple, entra.

c) Prototípica

Prototipo: “comida saludable” representada por ensalada/manzana.

- Más cercano: yogur natural
- Más lejano: pizza
- Borderline: pizza integral/“fit” con vegetales

Justificación: “saludable” no tiene frontera rígida (no hay propiedad única que lo determine). Se decide por semejanza al prototipo de “saludable”.




---

3) Conjunto: Dispositivos tecnológicos (celular, notebook, tablet, smartwatch, consola, e-reader)

a) Agrupamiento conceptual

Criterio: “para qué sirve” (intención del usuario)

- Comunicación: celular
- Productividad: notebook, tablet
- Entretenimiento: consola
- Lectura: e-reader
- Monitoreo personal: smartwatch

Justificación: es una clasificación por concepto/uso (descripción conceptual), no por una propiedad técnica única.

b) Categorización clásica

Reglas (propiedades técnicas):

- “Teléfono” = capacidad principal de llamadas/mensajería móvil → celular

- “Computadora” = ejecuta SO general + apps + I/O completa ⟶ notebook

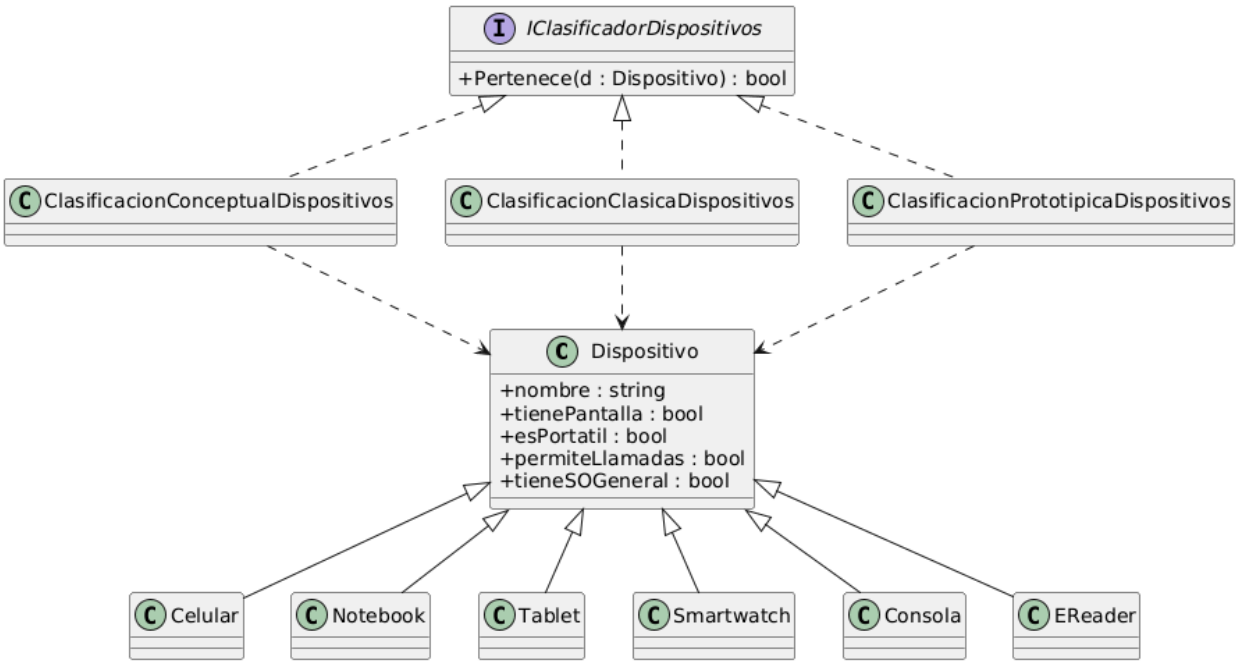
Justificación: son criterios técnicos que funcionan como condiciones para pertenecer o no pertenecer.

c) Prototípica

Prototipo: “computadora” ≈ notebook/PC.

- Similar: tablet con teclado
- Menos similar: celular
- Borderline: iPad Pro / Chromebook / consola con navegador

Justificación: algunos dispositivos se reconocen como “computadora” por parecido (uso, interacción), aunque no cumplan todos los rasgos. La frontera es difusa, no hay un límite exacto que separe quién pertenece y quién no. Se decide por grado de semejanza al prototipo.



# UNIDAD II

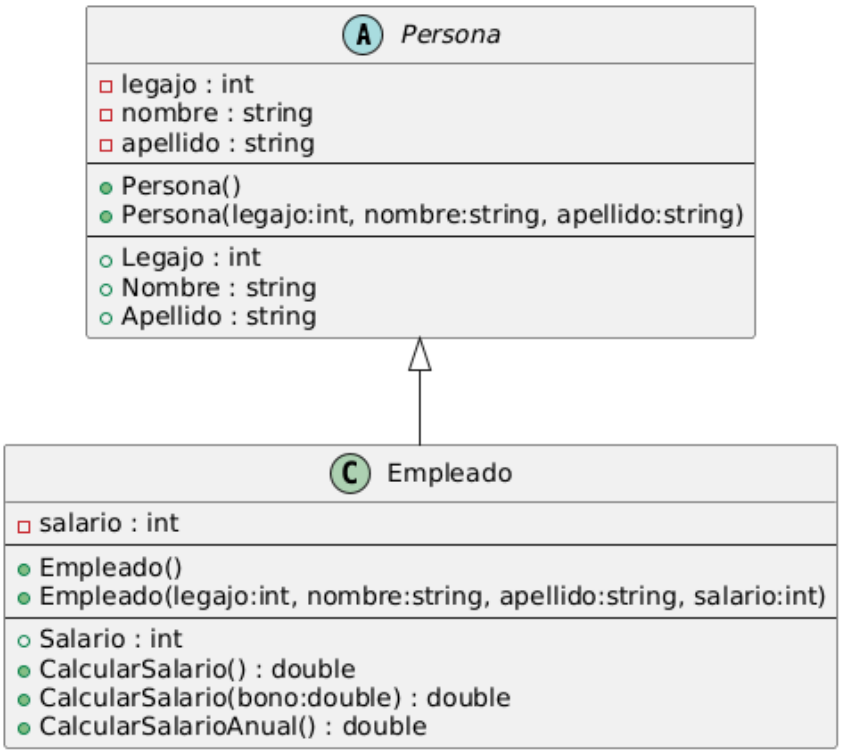
**Ejercicio 6**

Desarrollé un programa con una estructura de clases compuesta por una clase abstracta Persona y una clase derivada Empleado, aplicando el concepto de herencia.

Cumpliendo con la consigna, implementé la sobrecarga de constructores tanto en la clase Persona como en la clase Empleado. En Persona se definió un constructor por defecto Persona() y un constructor parametrizado Persona(int legajo, string nombre, string apellido). En Empleado también se implementó un constructor por defecto Empleado() al igual que un constructor parametrizado Empleado(int legajo, ...).

Por otra parte, en la clase Empleado se definieron dos métodos sobrecargados llamados CalcularSalario(): uno sin parámetros, que devuelve el salario base mensual, y otro que recibe un parámetro bono, permitiendo calcular el salario total sumando dicho adicional.

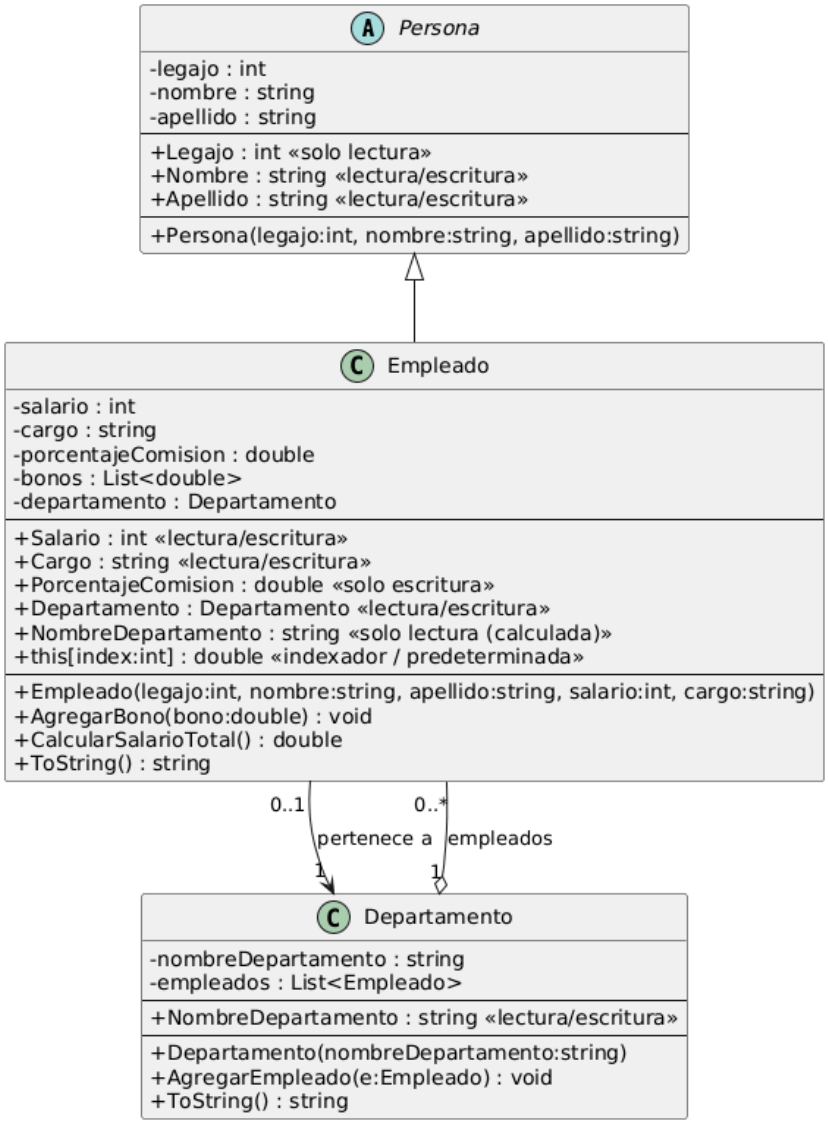
De esta manera, el programa cumple con el requerimiento de presentar una estructura de clases donde se observan constructores sobrecargados y métodos sobrecargados.



**Ejercicio 7**

En este programa, el formulario utiliza un DataGridView enlazado a una BindingList<Empleado>, lo que permite visualizar automáticamente las propiedades públicas que poseen método get. Esto resulta ser útil para evidenciar el tipo de propiedad implementada en cada caso:

- ➔ La propiedad Legajo (en Persona) es de solo lectura, por lo que puede visualizarse en la grilla pero no modificarse desde el exterior.
- ➔ La propiedad PorcentajeComision es de solo escritura, por lo que no aparece en el DataGridView, ya que no posee método get. Esto demuestra correctamente el concepto solicitado en el enunciado.
- ➔ Las propiedades como Salario, Cargo, Nombre y Apellido son de lectura-escritura, por lo que pueden visualizarse y modificarse.
- ➔ Los bonos se gestionan mediante un indexador (this[int index]), representando una propiedad con argumentos y a su vez una propiedad predeterminada en C#. Este tipo de propiedad no es interpretado como columna por el DataGridView, lo cual evidencia su naturaleza especial.

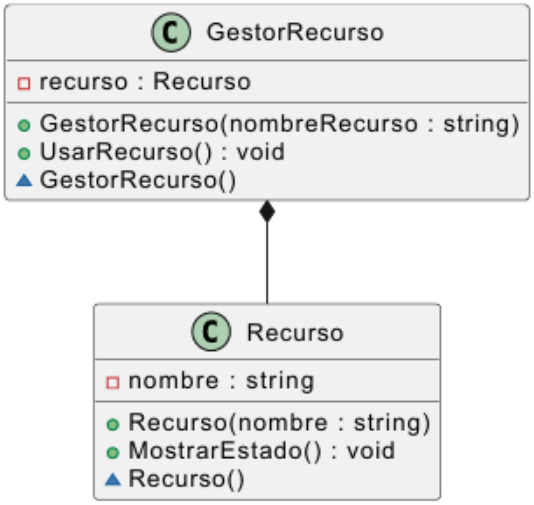


**Ejercicio 8**

Se implementaron las clases GestorRecurso y Recurso para representar el ciclo de vida de un objeto en C#.

La clase GestorRecurso instancia un objeto Recurso dentro de su constructor, estableciendo una relación de composición, ya que controla su creación y utilización.

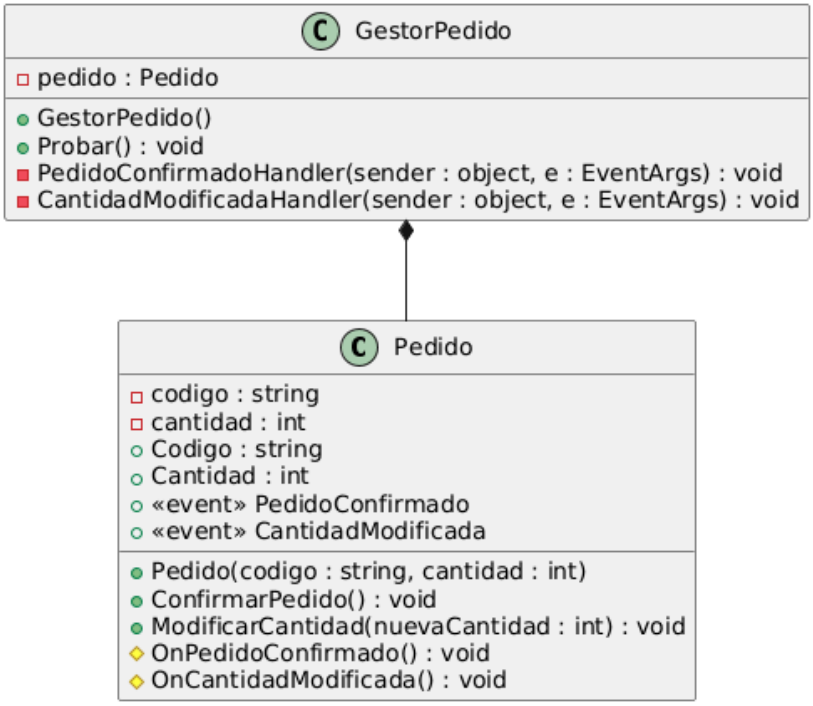
La clase Recurso implementa un finalizador (~Recurso) que se ejecuta cuando el objeto es recolectado por el *Garbage Collector*, permitiendo ejecutar código al finalizar su ciclo de vida.




**Ejercicio 9**

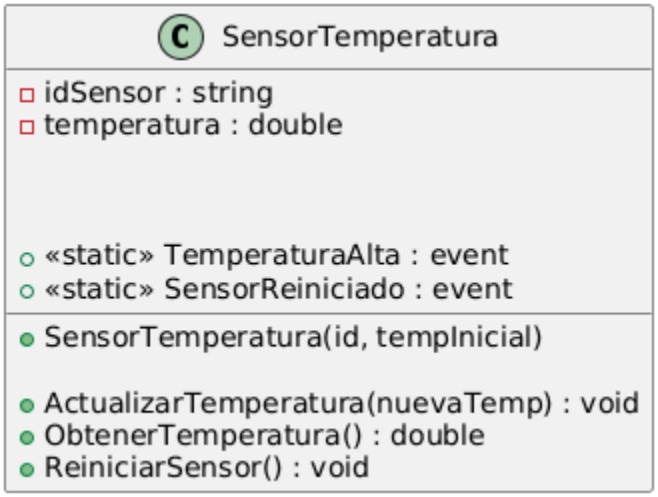
Se implementa la clase Pedido con propiedades, métodos y sucesos (eventos). Los eventos notifican cambios relevantes (confirmación y modificación de cantidad). La clase GestorPedido instancia un objeto Pedido y realiza suscripciones manuales (+=) a métodos propios de dicha clase, reaccionando a los sucesos mediante handlers.

Diagrama de Clases:



**Ejercicio 10**

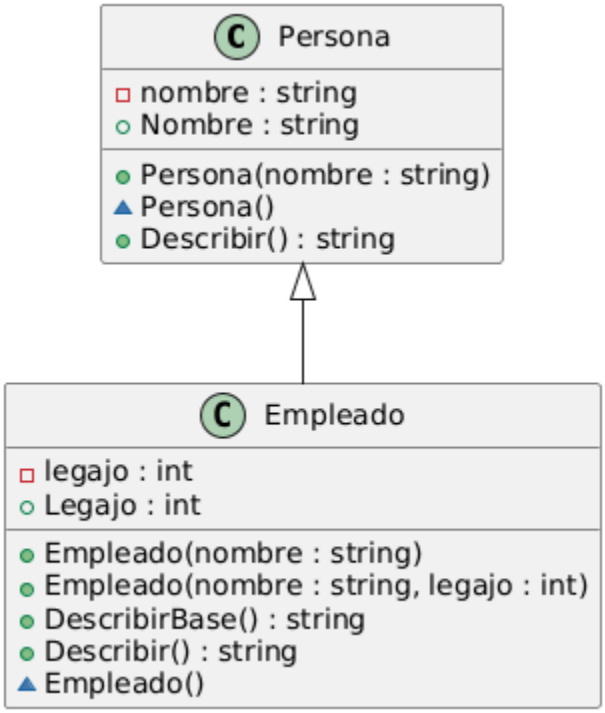
Se implementa la clase SensorTemperatura, la cual posee dos campos privados (idSensor y temperatura) que almacenan el estado interno del objeto. La clase incluye un constructor que inicializa dichos campos y tres métodos que permiten actualizar la temperatura, obtener su valor y reiniciar el sensor. Además, se definen dos sucesos estáticos (TemperaturaAlta y SensorReiniciado). Al ser estáticos, estos eventos pertenecen a la clase y pueden ser compartidos por todas las instancias, permitiendo notificar cambios relevantes del sistema.




**Ejercicio 11**

Se implementa una jerarquía de herencia entre Persona (base) y Empleado (derivada). En los constructores se utiliza base(nombre) para inicializar la parte heredada y this(nombre, legajo) para encadenar constructores dentro de la misma clase.

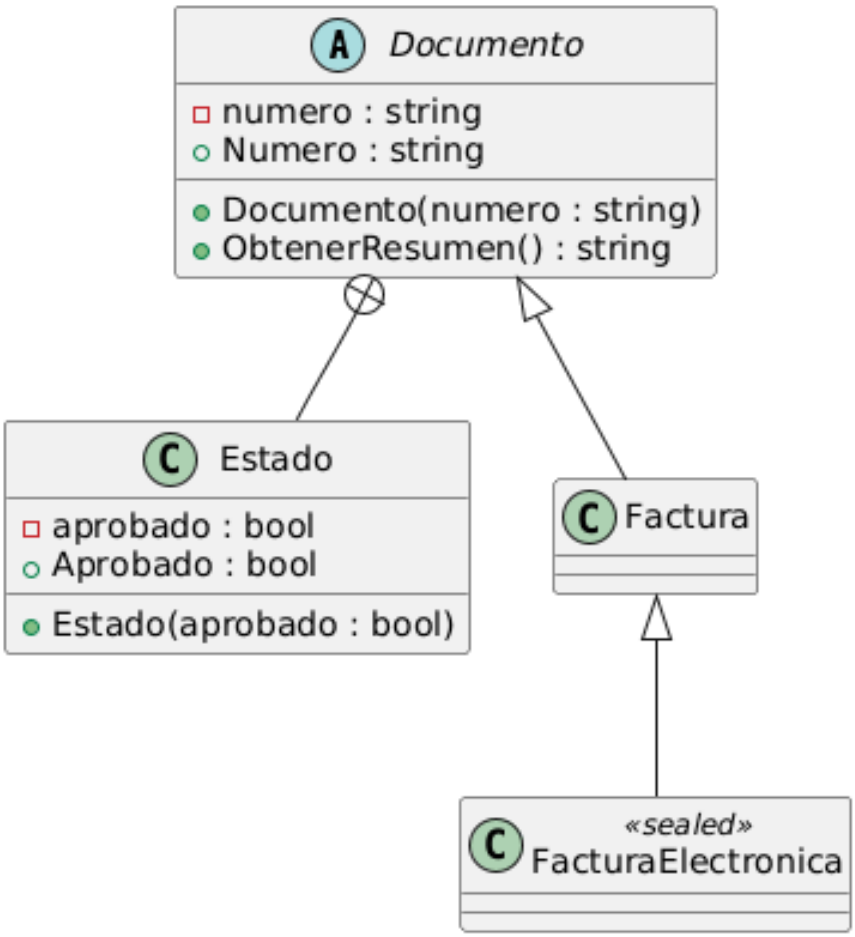
Se incluyen finalizadores en ambas clases para evidenciar el cierre del ciclo de vida de los objetos (no determinístico, a cargo del GC). Se demuestra también el uso de base para invocar explícitamente un miembro de la clase base y el uso de this para acceder a miembros de la instancia actual.



**Ejercicio 12**

Se implementa la clase abstracta Documento, que define un comportamiento común y contiene un método virtual ObtenerResumen() para permitir su redefinición en clases derivadas. Además, se incluye una clase anidada Estado dentro de Documento para representar información relacionada al documento. La clase Factura hereda de Documento y sobrescribe el método virtual.

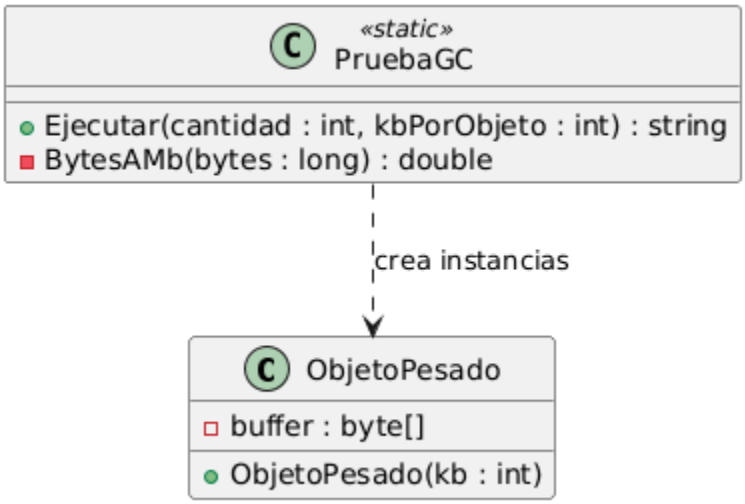
Finalmente, se implementa la clase FacturaElectronica como sellada (sealed), impidiendo que otras clases hereden de ella.



## UNIDAD III

**Ejercicio 13**

Al generar muchas instancias, la memoria administrada aumenta. Luego, al ejecutar el Garbage Collector, disminuye la memoria utilizada porque se liberan objetos sin referencias activas. Sin embargo, no siempre vuelve exactamente al valor inicial, ya que el runtime puede conservar memoria reservada para futuras asignaciones y los objetos con finalizadores o referencias vivas pueden demorar su liberación.



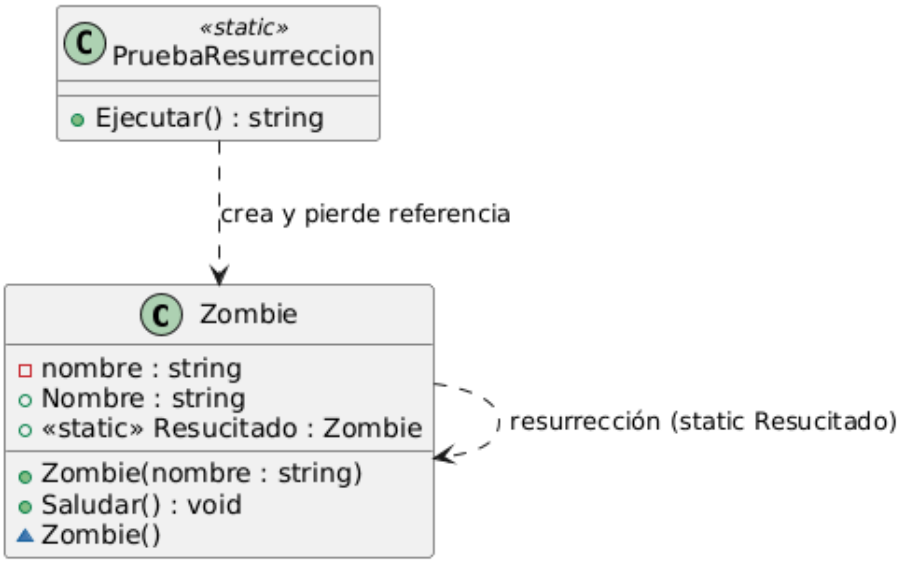

Se implementa la clase ObjetoPesado, que representa un objeto con consumo de memoria (buffer). La clase PruebaGC (estática) genera una cantidad importante de instancias de ObjetoPesado, mide la memoria antes y después de la creación, fuerza la ejecución del Garbage Collector y vuelve a medir. De esta manera se observa la liberación de memoria cuando los objetos quedan sin referencias activas.

**Ejercicio 14**

Se crea una instancia de un objeto y luego se pierde su referencia asignándola a null.

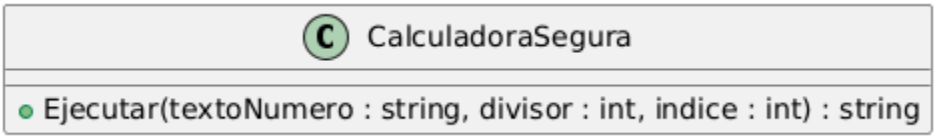
Cuando el Garbage Collector intenta liberar el objeto, el finalizador se ejecuta y guarda la referencia en un campo estático, permitiendo recuperar nuevamente el objeto.

De esta manera se observa la técnica de resurrección de objetos, donde un objeto que estaba por ser recolectado vuelve a quedar accesible en memoria.



**Ejercicio 15**

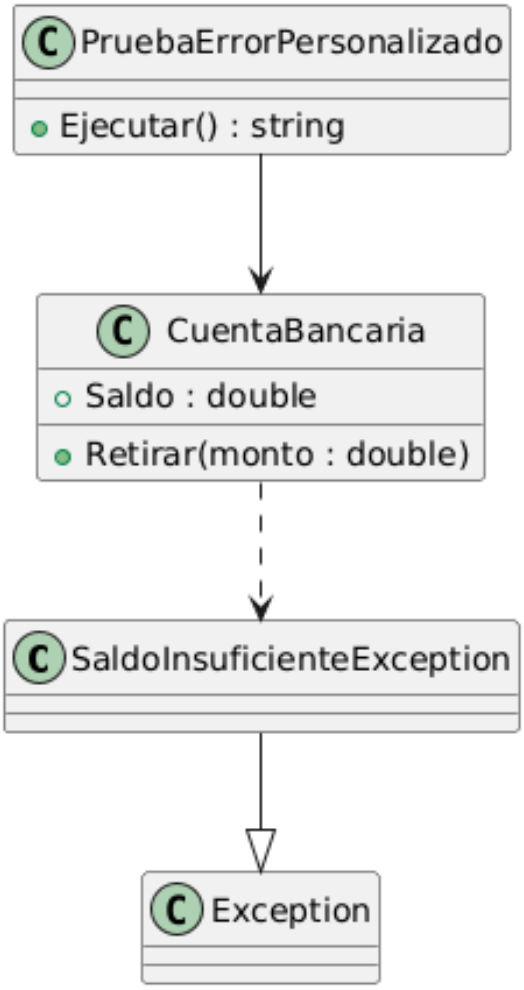
Se implementa una clase Calculadora que ejecuta distintas operaciones dentro de un bloque try. Cada operación puede generar diferentes excepciones que son capturadas mediante cinco bloques catch específicos: FormatException, DivideByZeroException, OverflowException, NullReferenceException e IndexOutOfRangeException. Finalmente se utiliza el bloque finally, el cual se ejecuta siempre independientemente de si ocurre o no una excepción, permitiendo realizar tareas de limpieza o cierre de recursos.




**Ejercicio 16**

Se implementa una excepción personalizada denominada SaldoInsuficienteException, la cual hereda de la clase base Exception.

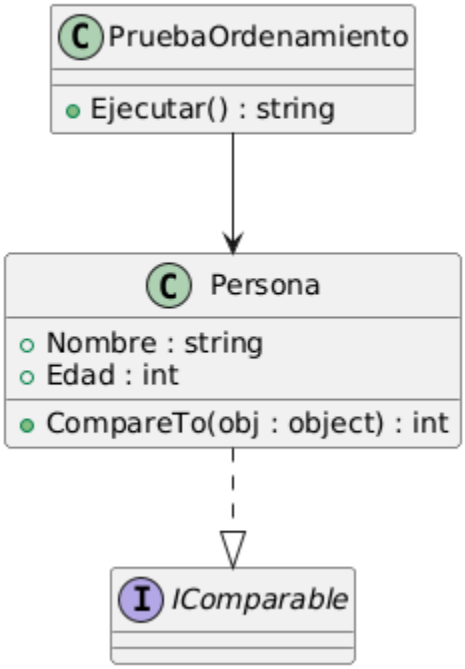
Esta excepción se dispara mediante la instrucción throw cuando se intenta retirar un monto mayor al saldo disponible en una cuenta bancaria. La excepción es capturada en un bloque catch, permitiendo mostrar un mensaje de error controlado al usuario.



## UNIDAD IV

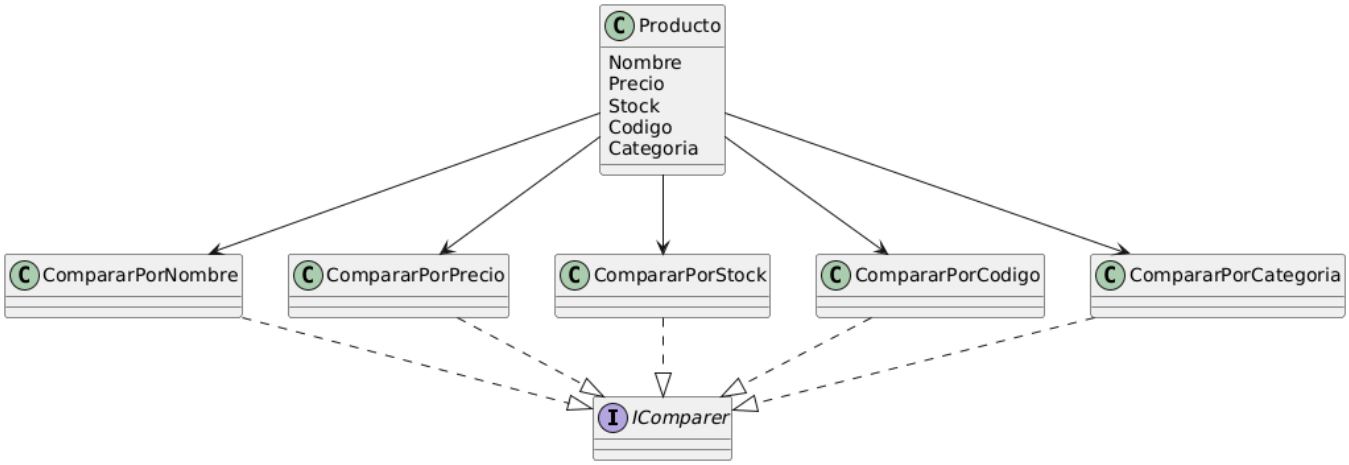
**Ejercicio 17**

Se implementa la interfaz IComparable en la clase Persona para definir el criterio de ordenamiento entre objetos. El método CompareTo() compara las edades de las personas y permite determinar el orden relativo entre ellas. Luego se genera un array de objetos Persona y se utiliza el método Array.Sort(), el cual invoca automáticamente el método CompareTo para ordenar los elementos.



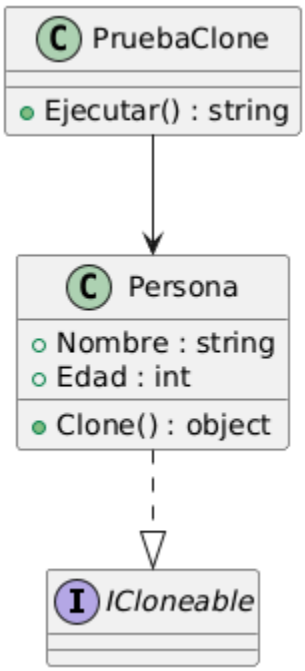

**Ejercicio 18**

Se implementan cinco clases comparadoras que implementan la interfaz IComparer, cada una definiendo un criterio de ordenamiento distinto para los objetos de la clase Producto: nombre, precio, stock, código y categoría. Luego se genera un array de objetos Producto y se utiliza el método Array.Sort() junto con cada comparador para ordenar los elementos según el criterio seleccionado.



**Ejercicio 19**

Se implementa la interfaz ICloneable en la clase PersonaEJ19, la cual define el método Clone() utilizado para generar una copia del objeto. En el programa se crea una instancia original y luego se genera su clon mediante el método Clone. Finalmente se demuestra que ambos objetos contienen los mismos datos pero no corresponden a la misma referencia en memoria utilizando el método ReferenceEquals().

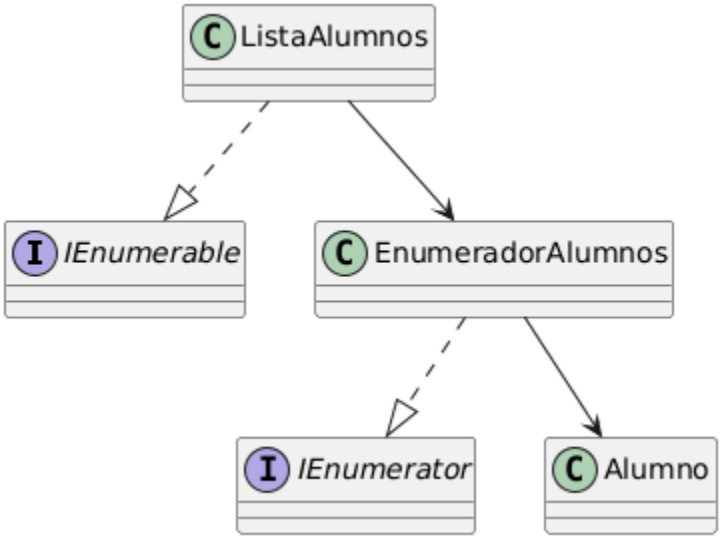



**Ejercicio 20**

Se implementa una colección personalizada denominada ListaAlumnos que implementa la interfaz IEnumerable, permitiendo recorrer sus elementos.

Para controlar el recorrido se implementa la interfaz IEnumerator en la clase EnumeradorAlumnos, la cual define los métodos MoveNext, Reset y la propiedad Current.

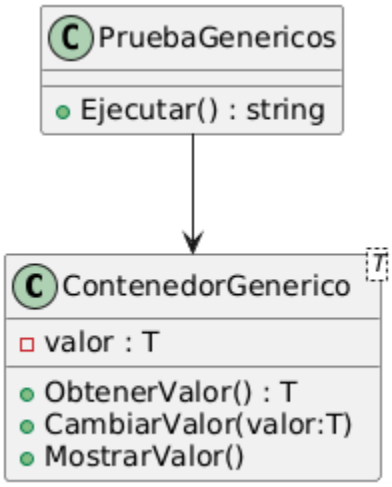
De esta manera es posible recorrer los objetos Alumno utilizando la estructura foreach.



## UNIDAD V

**Ejercicio 21**

Se implementa la clase genérica ContenedorGenerico<T>, la cual permite almacenar y manipular valores de cualquier tipo de dato. La clase define métodos que utilizan el tipo genérico para obtener y modificar el valor almacenado. De esta forma se demuestra el uso de genéricos a nivel de clase y métodos, permitiendo reutilizar la misma estructura para distintos tipos de datos como enteros o cadenas.

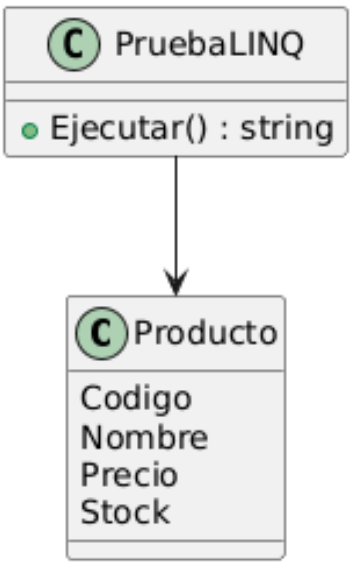



**Ejercicio 22**

Se utiliza LINQ (Language Integrated Query) para realizar consultas sobre una colección de objetos de la clase Producto.

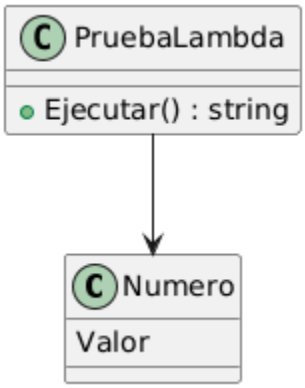
Se realizan diferentes consultas utilizando las cláusulas from, where, select y orderby, permitiendo filtrar, ordenar y proyectar información de la lista.

Los resultados se recorren mediante estructuras foreach para mostrar la información obtenida.



**Ejercicio 23**

Se utilizan expresiones lambda para definir funciones anónimas de forma compacta dentro de consultas sobre colecciones. En el programa se aplican lambdas para filtrar, ordenar y seleccionar datos de una lista de objetos mediante métodos de LINQ como Where, OrderBy y Select. Esto permite escribir código más conciso y legible.

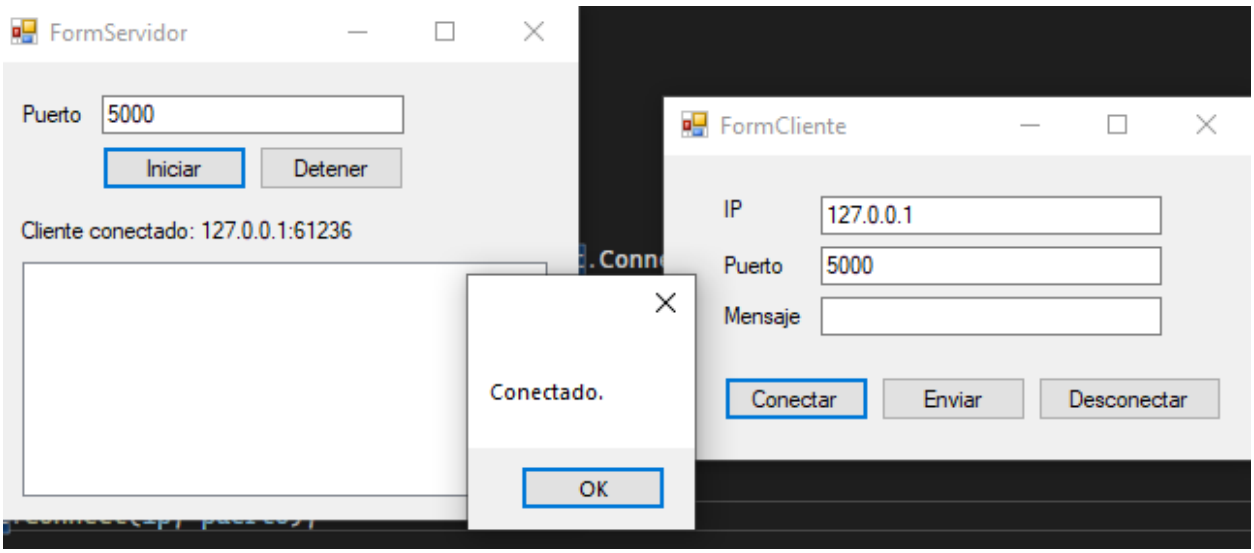


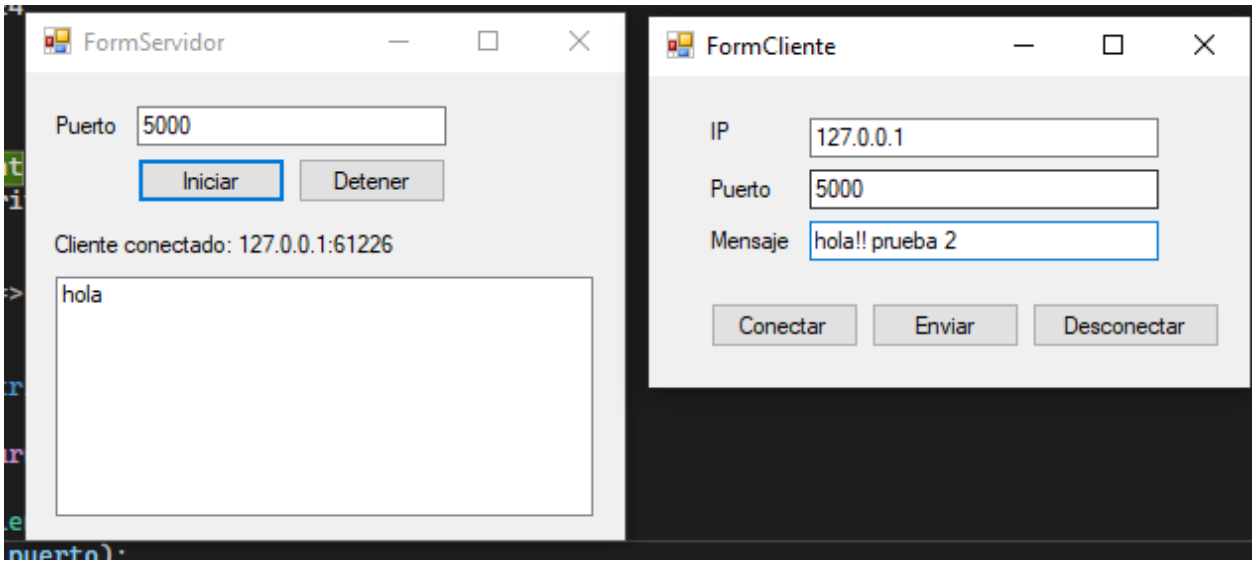

## UNIDAD VI

**Ejercicio 24**

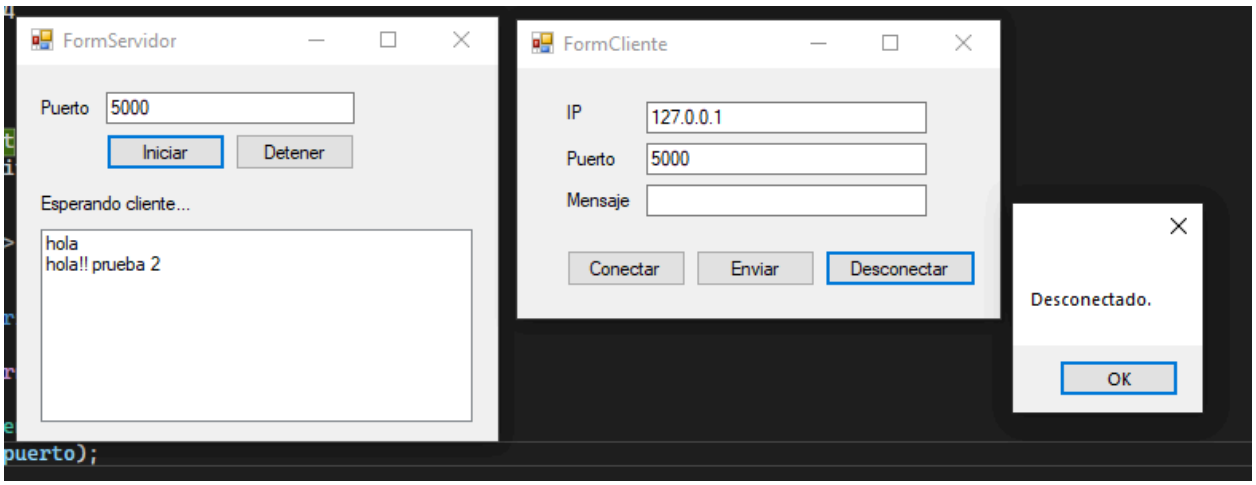
Se implementa una comunicación en red mediante sockets TCP. La clase SocketServidor escucha en un puerto, acepta una conexión y recibe mensajes de texto, notificando a la interfaz a través del evento MensajeRecibido. La clase SocketCliente se conecta a la IP/puerto del servidor y envía mensajes con WriteLine, que el servidor lee mediante ReadLine. En WinForms se prueban ambas aplicaciones: el cliente escribe un texto y el servidor lo visualiza en tiempo real.

Paso a paso en el código:






Desconectamos:



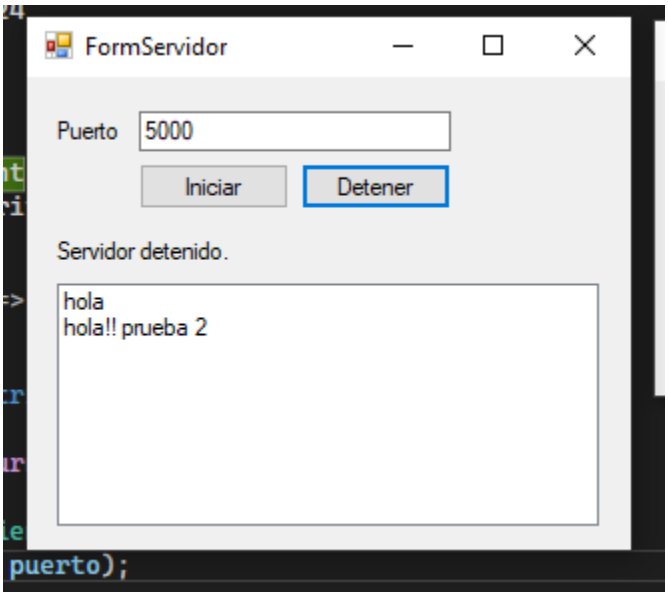
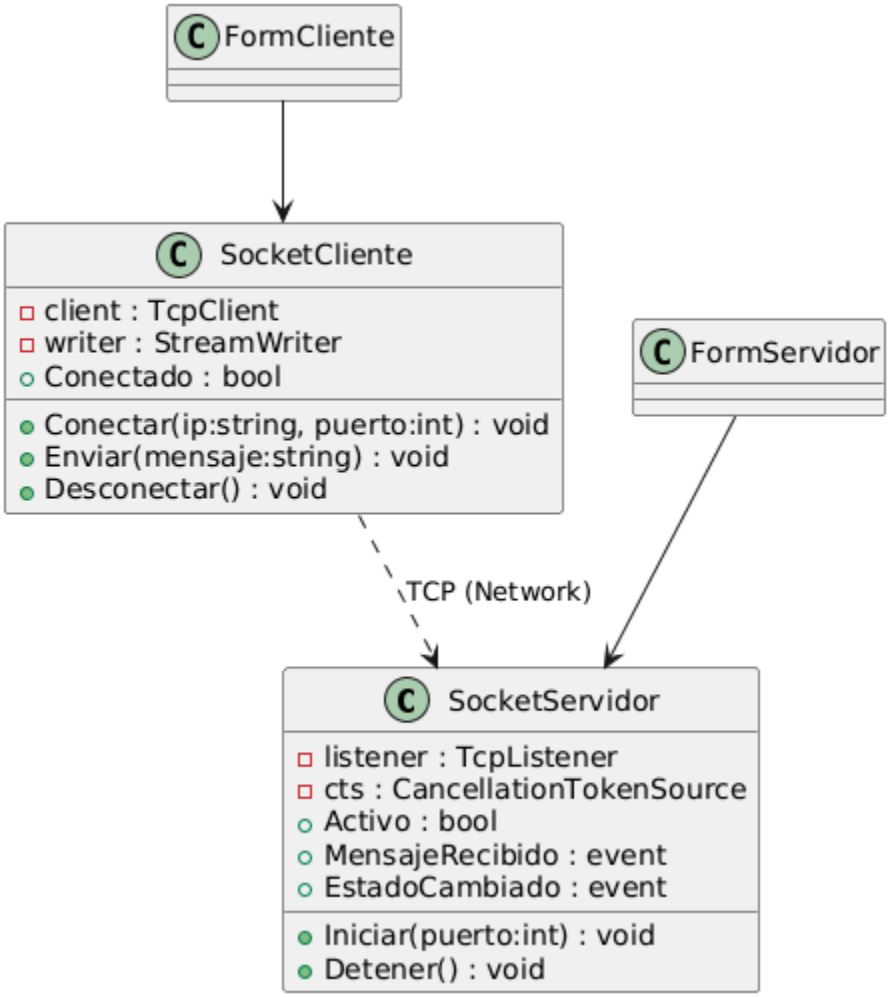



Diagrama de Clases:

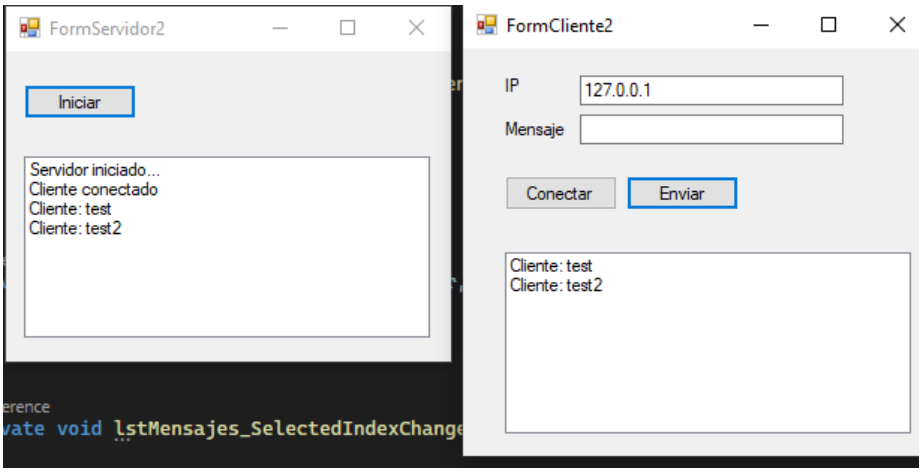



**Ejercicio 25**

El sistema se compone de dos interfaces WinForms (FormServidor2 y FormCliente2) que interactúan con las clases de lógica ChatServer y ChatClient.

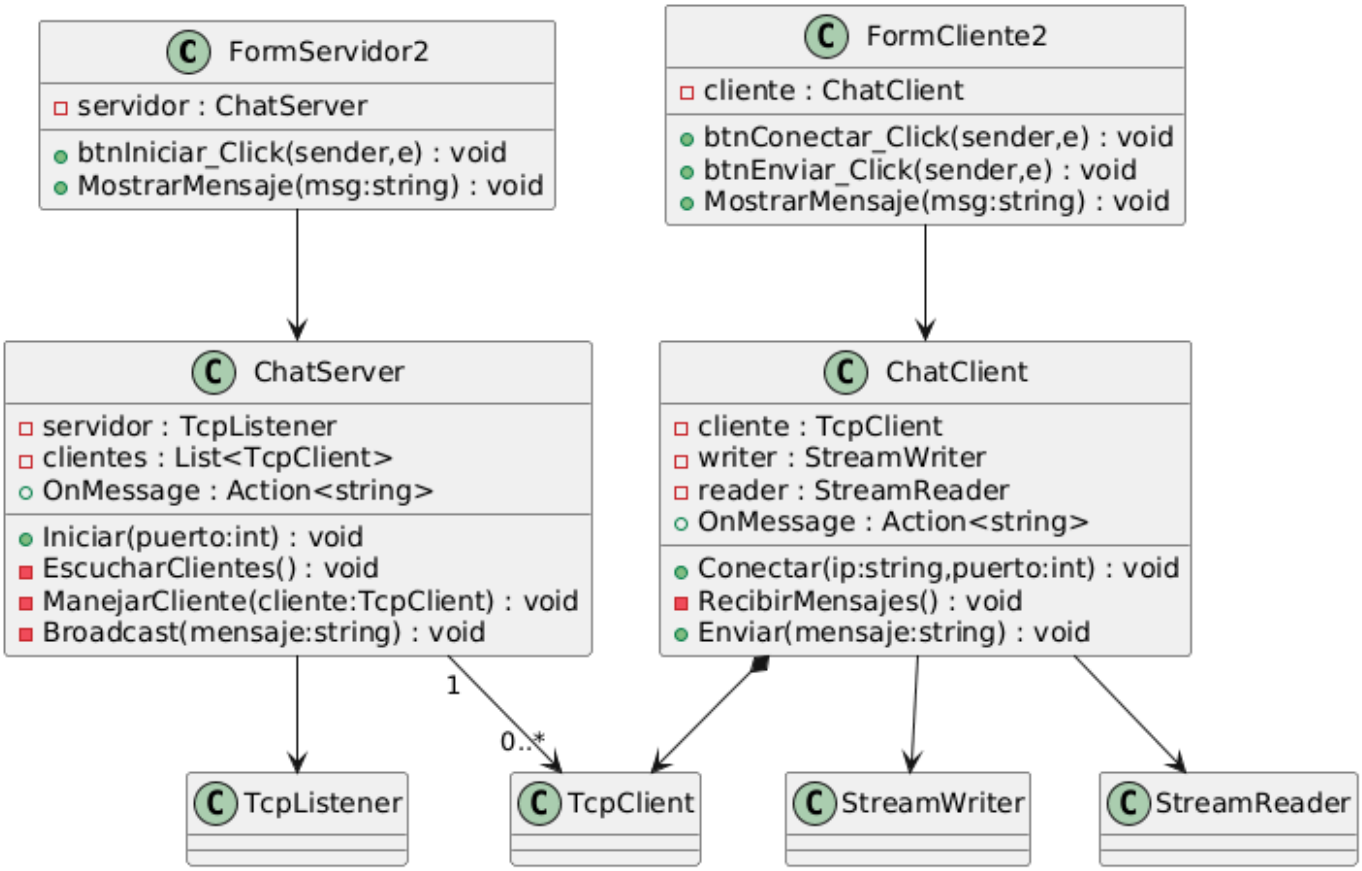
El servidor utiliza TcpListener para aceptar conexiones y mantiene una colección de TcpClient.

Cada cliente se conecta al servidor



mediante TcpClient, utilizando StreamWriter y StreamReader para enviar y recibir mensajes.

El servidor recibe mensajes y los redistribuye mediante un mecanismo de broadcast a todos los clientes conectados.

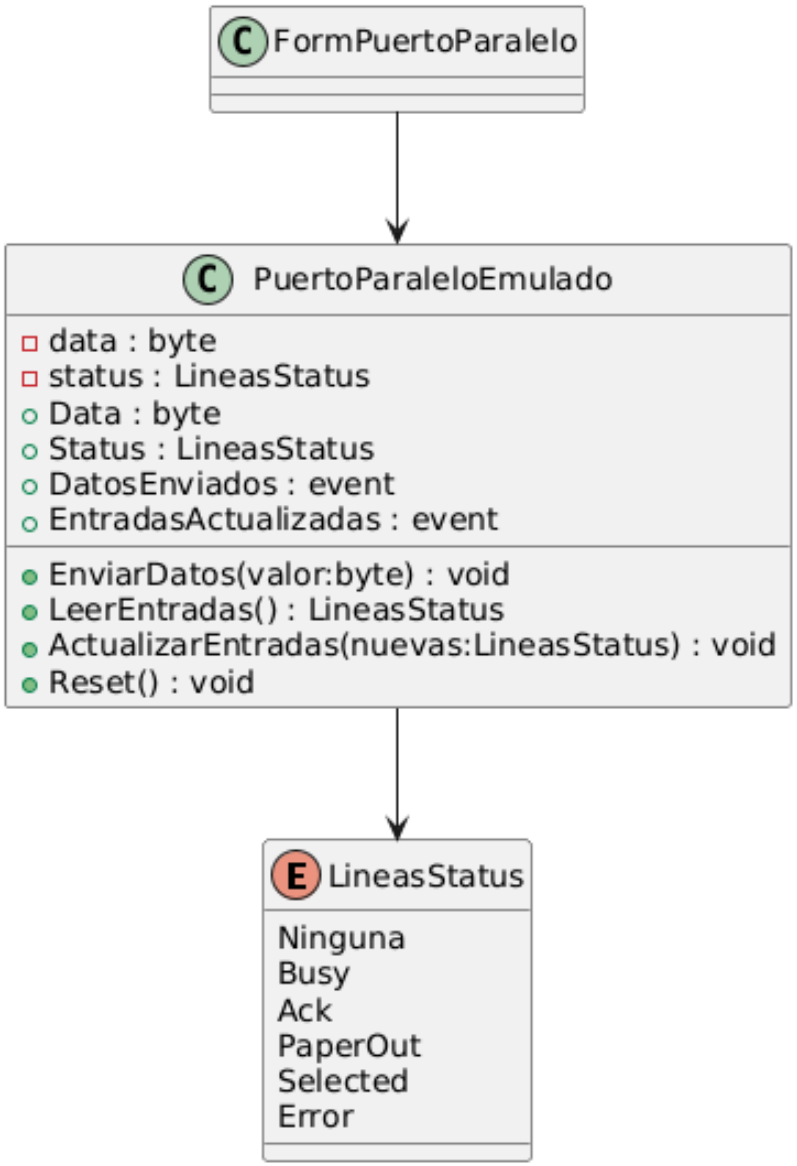



**Ejercicio 26**

Se implementa un emulador de puerto paralelo que modela el registro DATA (salida de 8 bits) y el registro STATUS (entradas). El programa permite enviar señales escribiendo un byte en DATA y luego recolectar entradas leyendo el estado del registro STATUS.

El emulador genera respuestas simuladas del dispositivo y notifica cambios mediante eventos, permitiendo visualizar en la interfaz el flujo *"envío → respuesta → lectura".*

Diagrama de Clases:



# GUÍA DE ABORDAJE BIBLIOGRÁFICO

## UNIDAD I

Cardacci Dario y Booch, Grady. **Orientación a Objetos. Teoría y Práctica.** -- Buenos Aires, Argentina. Pearson Argentina, 2013. Capítulos 1 al 4.

Deitel Harvey M. Y Paul J. Deitel. **Cómo programar en C#**. Segunda edición. Pearson. México 2007. Capítulos 9 al 11

## UNIDAD II

Cardacci Dario y Booch, Grady. **Orientación a Objetos. Teoría y Práctica.** -- Buenos Aires, Argentina. Pearson Argentina, 2013. Capítulos 5.1 al 5.6

Deitel Harvey M. Y Paul J. Deitel. **Cómo programar en C#**. Segunda edición. Pearson. México 2007. Capítulos 9 al 11

## UNIDAD III

https://docs.microsoft.com/es-es/dotnet/csharp/language-reference/keywords/exception-handling-statements

Balena, Francesco. **Programación avanzada con Visual Basic 2005.** -- México, DF: McGraw-Hill Interamericana, c2008. Capítulo 1.

Deitel Harvey M. Y Paul J. Deitel. **Cómo programar en C#**. Segunda edición. Pearson. México 2007. Capítulos 12.

## UNIDAD IV

https://docs.microsoft.com/es-es/dotnet/csharp/tour-of-csharp/interfaces

https://docs.microsoft.com/es-es/dotnet/csharp/tour-of-csharp/delegates

## UNIDAD V

Deitel Harvey M. Y Paul J. Deitel. **Cómo programar en C#**. Segunda edición. Pearson. México 2007. Capítulos 25

https://docs.microsoft.com/es-es/dotnet/csharp/programming-guide/concepts/linq/

https://docs.microsoft.com/es-es/dotnet/csharp/programming-guide/generics/

https://docs.microsoft.com/es-es/dotnet/csharp/programming-guide/concepts/linq/linq-and-generic-types

https://docs.microsoft.com/es-es/dotnet/csharp/programming-guide/statements-expressions-operators/lambda-expressions

## UNIDAD VI

MSDN. Microsoft Developer Network.
http://msdn.microsoft.com/en-us/library/system.net.sockets.socket.aspx

Cardacci Dario y Booch, Grady. **Orientación a Objetos. Teoría y Práctica.** -- Buenos Aires, Argentina. Pearson Argentina, 2013. Capítulos 5.7.

Deitel Harvey M. Y Paul J. Deitel. **Cómo programar en C#**. Segunda edición. Pearson. México 2007. Capítulos 23

Montefinal, Fabián H.; Cardacci, Darío G. **Conceptos básicos sobre electricidad: electrónica y puertos de la PC**. 2a.ed.-- Buenos Aires: Universidad Abierta Interamericana, c2006. 78 páginas